

Assignment 3 - Sudarshan V

Question 1

Consider the following code and choose the odd one out

Code

```
for i in range(1,5):  
    print(i**2)
```

Choices - a) 16 b) 25 c) 1 d) 4 e) 9

```
In [2]: print("Question 1 - Actual Testing")  
  
for i in range(1,5):  
    print(i**2)
```

Question 1 - Actual Testing

1
4
9
16

Question 1 : Answer b) 25 is the odd

Question 2

Prime number are the unique set of numbers widely used in mathematics.

It is because of the fact that the number is divisible by 1 and itself.

Now construct a function that determines the whether the given is prime or not and

further reuse this function to print all the prime number between 1 and the N(like if N=10 prime numbers are 2,3,5,7)

```
In [43]: # Question 2 - Prime Number Function  
  
def chk_prime(num_in):  
    if num_in < 2:  
        return False  
  
    for i in range(2, int(num_in**0.5)+1):  
        if num_in % i == 0:  
            return False  
  
    return True  
  
while True:  
    My_Num = int(input("Enter the number between 1 and N to display prime number Ra  
    if My_Num > 0:  
        break
```

```

else :
    print("Enter Number > 1")
    continue

#My_Num = 7

print("If N = ", My_Num, "The Prime Numbers are", end=": ")

for i in range(1, My_Num+1):
    if chk_prime(i):
        print(i, end=" ", )

```

If N = 33 The Prime Numbers are: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,

Question 3

What is the output of the following code

Code

```

x = [1, 2, 3, 4, 5]
print(x[1:4])

```

Choices

- a) [1, 2, 3]
- b) [2, 3, 4]
- c) [1, 2, 3, 4]
- d) [2, 3, 4, 5]

In [44]: # Question 3 Program

```

x = [1, 2, 3, 4, 5]
print(x[1:4])

```

[2, 3, 4]

Question 3 Answer b) [2, 3, 4]

Question 4

Which of the following is TypeError?

Code

```

a = "5"
b = 2

```

Choices

- a) a*b
- b) int(a) + b
- c) a + str(b)
- d) a + b

```
In [53]: a = "5"
b = 2

test1 = a*b # Gets Duplicated twice
print("a", test1)
test2 = int(a) + b # Typecasted - Hence work
print("b", test2)
test3 = a + str(b) # Typecasted - Hence works
print("c", test3)
# test4 = a + b # this is an issue because str and Integer addition does not happen
print("d", "error")
```

a 55
b 7
c 52
d error

Question 4 Answer d) a + b

Question 5

What does the below code print.?

Code

```
for i in range(3):
    pass
print(i)
```

Choices

- a) Nothing - i is undefined
- b) 2
- c) 3
- d) 0

```
In [54]: # Question 5 - Try Out

for i in range(3):
    pass
print(i)
```

2

Question 5 Answer b) 2

Question 6

What would be the output data types?

Code

```
print(3/2)
print(3//2)
```

Choices

- a) int, int
- b) float, int
- c) float, float
- d) int, float

```
In [56]: # Question 6 - Try Out
print(3/2) # My Understanding - this is normal division
print(3//2) # This is floor division (remove the part post decimal)

# Better example for my understanding
print(10/3)
print(10//3)
```

```
1.5
1
3.3333333333333335
3
```

Question 6 Answer b) float, int

Question 7

What would be the output of the code below

Code

```
x=10
def foo():
    x= 20
    print(x)
```

```
foo()
print(x)
```

Choices

- a) 20, 20
- b) 10, 10
- c) 20, 10
- d) error

```
In [57]: # Question 7 - Code Tryout

x=10
def foo():
    x= 20
    print(x)

foo()
print(x)
```

```
20
10
```

Question 7 Answer c) 20, 10

Question 8

what will be the output.?

Code

```
print(bool(0), bool(''), bool([]))
print(bool(-1), bool('0'), bool([0]))
```

Choices

- a) False, True
- b) False, True, False, True, False, True
- c) False, False, False, True, True, True
- d) True, False, False, False, True, True

In [58]: *# Question 8 - Code try out*

```
print(bool(0), bool(''), bool([]))
print(bool(-1), bool('0'), bool([0]))
```

False False False
True True True

In [59]: *##### Question 8 Answer c) False, False, False, True, True, True*

Question 9

Consider the following code and identify

what is the difference between // and % operator and how they both are different.

What is the correct output for the code below.?

Code

```
print(3//2)
print(3%2)
print(7//3)
print(7%3)
```

Choices

- a) 1 1 2 1
- b) 1 1 1 1
- c) 1 2 1 2
- b) 2 1 1 2

In [60]: *# Question 9 - Code Try Out*

```
print(3//2)
# My understanding - Floor operator - 1.5 floored to 1

print(3%2)
# My understanding - Mod operator - 3 MOD 2 , Reminder is 1

print(7//3)
```

```
# My understanding - Floor operator - 2.3333 floored to 2

print(7%3)
# My understanding - Mod operator - 7 MOD 3 , Reminder is 1
```

```
1
1
2
1
```

In [61]: ##### Question 9 Answer a) 1 1 2 1

Question 10

You are given a dataset containing student information in a raw and inconsistent format.

Your task is to clean the data and perform basic analysis

Tasks

1. Data Cleaning

- Remove all invalid records (e.g., missing values, "NA", non-numeric marks or age).
- Convert marks and age into integers.
- Store the cleaned data in a suitable structure.

2. Calculate the Average Marks after cleaning the data.

3. Display all the names of student whose marks are even.

4. Display the name the topper

5. Allocated different batches to each student with each batch size of at least 2 and display the same.

Examples :-

Batch 0 → first 2 students

Batch 1 → next 2 students

Dataset

Input Data:

```
raw_data = [
    "Rahul,78,20",
    "Anita,85,21",
    "John,NA,19",
    "Meena,92,20",
    "Kiran,,22",
    "Arjun,67,twenty",
    "Divya,88,21"
]
```

In [118... # Question 10 - Program

```

# Planning to use pandas and numpy

import pandas as pd
import numpy as np

# Given Data
raw_data = [
    "Rahul,78,20",
    "Anita,85,21",
    "John,NA,19",
    "Meena,92,20",
    "Kiran,,22",
    "Arjun,67,twenty",
    "Divya,88,21"
]

print("\nRaw Data:\n\n", raw_data)

# Format the data !

split_data_r = []

for x in raw_data:
    x_temp = x.split(",")
    split_data_r.append(x_temp)

print("\nFirst Level Formatting of Data : \n")
print(split_data_r) # this has become 2 DIM Array now

# Formated data is available now

# Using pandas create a dataframe

MyDF = pd.DataFrame(split_data_r, columns=["Name", "Marks", "Age"])

print("\nFormatted Data -> Pandas Dataframe\n ----- \n", MyDF)

# Step 1 - Data Cleaning
#         • Remove all invalid records (e.g., missing values, "NA", non-numeric marks)
#         • Convert marks and age into integers.
#         • Store the cleaned data in a suitable structure.

# Step 1a : Check for errors and add NaN!

MyDF["Marks"] = pd.to_numeric(MyDF["Marks"], errors='coerce')

print("\n Marks Corrected\n", MyDF)

MyDF["Age"] = pd.to_numeric(MyDF["Age"], errors='coerce')

print("\n Age Corrected\n", MyDF)

# Step 1b : Remove NaN !! # Step 1c - Stored the clean data in MyCDF Structure !

```

```

MyCDF = MyDF.dropna()
MyCDF = MyCDF.reset_index(drop=True)  # Did this after getting error !

print("\nCleaned Data in New Structure\n", MyCDF)

# Convert float to int - Discovered that Pandas was giving error!
MyCDF["Marks"] = MyCDF["Marks"].astype(int)
MyCDF["Age"] = MyCDF["Age"].astype(int)

# Step 2 - Average Marks Calculation !

avg_Marks = MyCDF["Marks"].mean()

print("\nAverage Marks :", avg_Marks)

# Step 3 - Even Marks Scorers in the List

#for i in range(len(MyCDF)):
#    if MyCDF["Marks"] % 2 == 0:
#        print(MyCDF["Name"][i], MyCDF["Marks"][i], MyCDF["Age"][i])

print("\nEven Mark Scorers in the List : \n")

for i in range(len(MyCDF)):

    if MyCDF["Marks"][i] % 2 == 0:

        print(f"Name = {MyCDF['Name'][i]}, "
              f"Marks = {MyCDF['Marks'][i]}, "
              f"Age = {MyCDF['Age'][i]}")

# Step 4 - Name of the topper

Topper_Marks = MyCDF["Marks"].iloc[0]

Topper_Index = 0

for i in range(len(MyCDF)):

    if MyCDF["Marks"].iloc[i] > Topper_Marks:
        Topper_Marks = MyCDF["Marks"].iloc[i]
        Topper_Index = i

print(f"\nTopper in the list is : {MyCDF['Name'].iloc[Topper_Index]}")

# Step 5 - Name of the topper

batch_size = 2

for i in range(0, len(MyCDF), batch_size):
    batch = MyCDF.iloc[i:i+batch_size]

```



```
print(f"\nBatch : {i//batch_size}")  
for name in batch["Name"]:  
    print("    ", name)
```

Raw Data:

```
['Rahul,78,20', 'Anita,85,21', 'John,NA,19', 'Meena,92,20', 'Kiran,,22', 'Arjun,67,
twenty', 'Divya,88,21']
```

First Level Formating of Data :

```
[['Rahul', '78', '20'], ['Anita', '85', '21'], ['John', 'NA', '19'], ['Meena', '92',
'20'], ['Kiran', '', '22'], ['Arjun', '67', 'twenty'], ['Divya', '88', '21']]
```

Formatted Data -> Pandas Dataframe

	Name	Marks	Age
0	Rahul	78	20
1	Anita	85	21
2	John	NA	19
3	Meena	92	20
4	Kiran		22
5	Arjun	67	twenty
6	Divya	88	21

Marks Corrected

	Name	Marks	Age
0	Rahul	78.0	20
1	Anita	85.0	21
2	John	NaN	19
3	Meena	92.0	20
4	Kiran	NaN	22
5	Arjun	67.0	twenty
6	Divya	88.0	21

Age Corrected

	Name	Marks	Age
0	Rahul	78.0	20.0
1	Anita	85.0	21.0
2	John	NaN	19.0
3	Meena	92.0	20.0
4	Kiran	NaN	22.0
5	Arjun	67.0	NaN
6	Divya	88.0	21.0

Cleaned Data in New Structure

	Name	Marks	Age
0	Rahul	78.0	20.0
1	Anita	85.0	21.0
2	Meena	92.0	20.0
3	Divya	88.0	21.0

Average Marks : 85.75

Even Mark Scorers in the List :

Name = Rahul, Marks = 78, Age = 20

Name = Meena, Marks = 92, Age = 20

Name = Divya, Marks = 88, Age = 21

Topper in the list is : Meena

Batch : 0
Rahul
Anita

Batch : 1
Meena
Divya

In []: